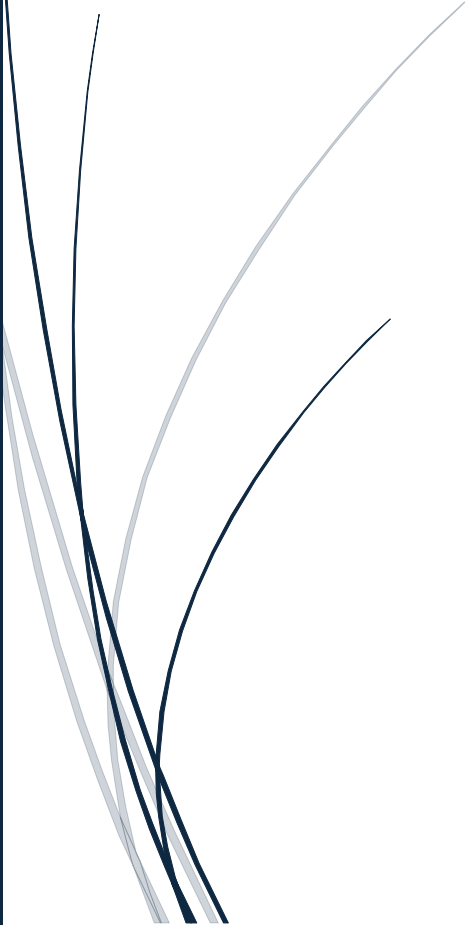




21/10/2025

Esercizio facoltativo Shell Linux

A cura di Fabio Renna



Titolo: Esercizi facoltativo Shell Linux

Autore: Fabio Renna

Data: 21/10/2025

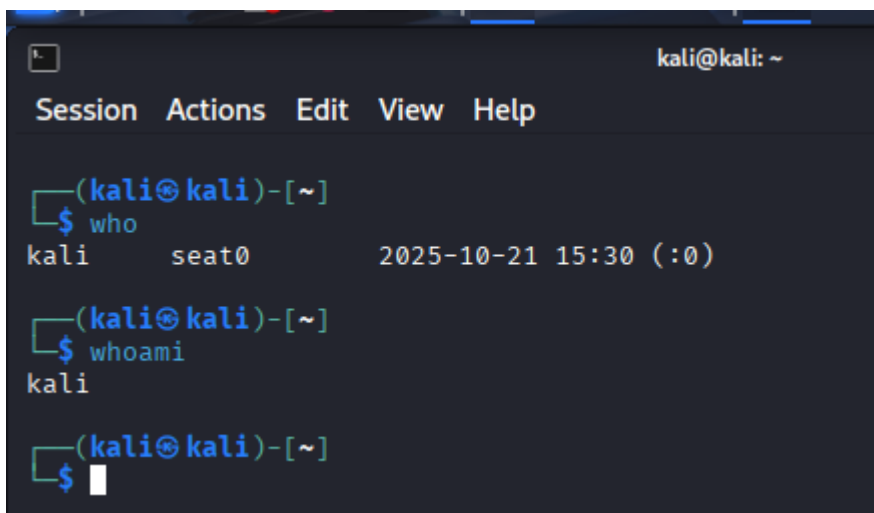
Ambiente di lavoro: Kali Linux

Obiettivo:

Familiarizzare con i comandi base per la gestione dei processi (ps, jobs, kill, bg, fg) e comprendere la differenza tra foreground e background.

Procedura:

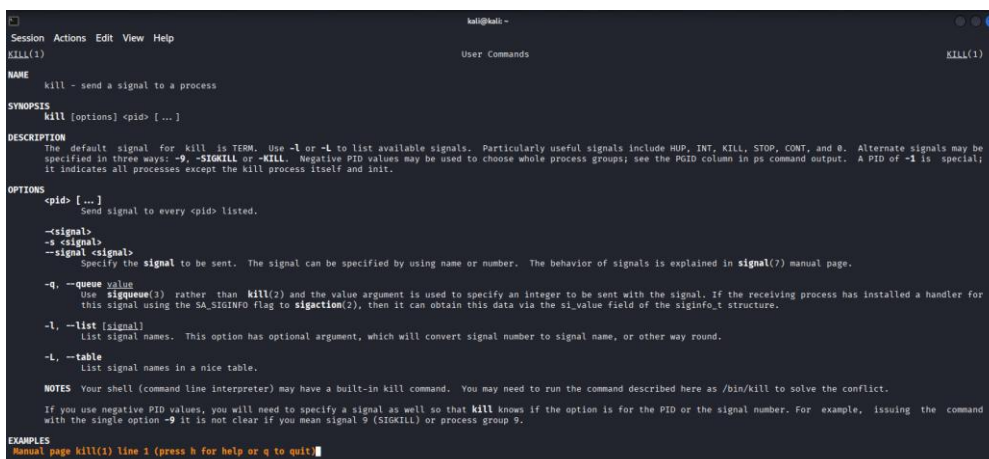
Inizialmente siamo andati ad utilizzare i comandi “who” e “whoami” per identificare gli utenti collegati al sistema:



```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ who  
kali    seat0      2025-10-21 15:30 (:0)  
(kali@kali)-[~]  
$ whoami  
kali  
(kali@kali)-[~]  
$
```

L’esercizio facoltativo di oggi, poi, ci chiede di aprire il terminale e leggere i manuali dei comandi “**jobs**” – “**ps**” – “**kill**”

Per i manuali di “**ps**” e “**kill**” abbiamo utilizzato il comando “**man**” che sta appunto per Manuale.



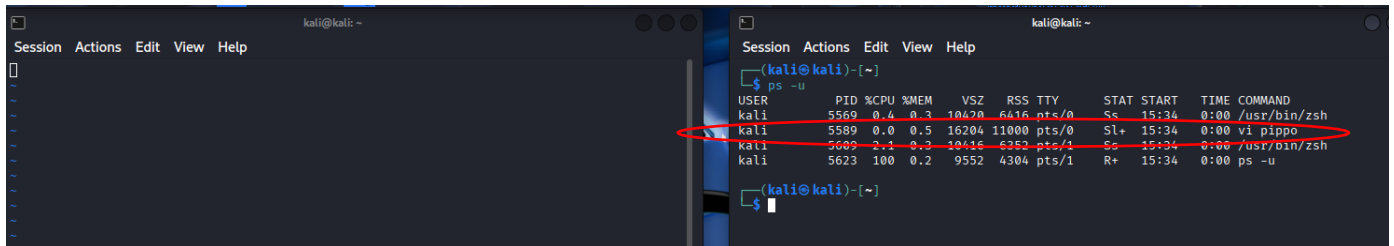
```
kill(1) User Commands kill(1)  
NAME  
kill - send a signal to a process  
SYNOPSIS  
kill [options] <pid> [...]  
DESCRIPTION  
The default signal for kill is TERM. Use -l or -L to list available signals. Particularly useful signals include HUP, INT, KILL, STOP, CONT, and 0. Alternate signals may be specified in three ways: -9, -SIGKILL or -KILL. Negative PID values may be used to choose whole process groups; see the PGID column in ps command output. A PID of -1 is special; it indicates all processes except the kill process itself and init.  
OPTIONS  
<pid> [...]  
Send signal to every <pid> listed.  
-s, --signal=signal  
Specify the signal to be sent. The signal can be specified by using name or number. The behavior of signals is explained in signal(7) manual page.  
-q, --queue=queue  
Use sigqueue(3) rather than kill(2) and the value argument is used to specify an integer to be sent with the signal. If the receiving process has installed a handler for this signal using the SA_SIGINFO flag to sigaction(2), then it can obtain this data via the si_value field of the siginfo_t structure.  
-l, --list=[signal]  
List signal names. This option has optional argument, which will convert signal number to signal name, or other way round.  
-L, --table  
List signal names in a nice table.  
NOTES  
Your shell (command line interpreter) may have a built-in kill command. You may need to run the command described here as /bin/kill to solve the conflict.  
If you use negative PID values, you will need to specify a signal as well so that kill knows if the option is for the PID or the signal number. For example, issuing the command with the single option -9 it is not clear if you mean signal 9 (SIGKILL) or process group 9.  
EXAMPLES  
Manual page kill(1) line 1 (press h for help or q to quit)
```

```
kali@kali: ~  
Session Actions Edit View Help  
PS(1) User Commands PS(1)  
NAME  
ps - report a snapshot of the current processes.  
SYNOPSIS  
ps [options]  
DESCRIPTION  
ps displays information about a selection of the active processes. If you want a  
repetitive update of the selection and the displayed information, use top instead.  
This version of ps accepts several kinds of options:  
1 UNIX options, which may be grouped and must be preceded by a dash.  
2 BSD options, which may be grouped and must not be used with a dash.  
3 GNU long options, which are preceded by two dashes.  
Options of different types may be freely mixed, but conflicts can appear. There are  
some synonymous options, which are functionally identical, due to the many standards  
and ps implementations that this ps is compatible with.  
By default, ps selects all processes with the same effective user ID (euid=EUID) as  
the current user and associated with the same terminal as the invoker. It displays  
the process ID (pid=PID), the terminal associated with the process (tname=TTY), the  
cumulated CPU time in [DD-]hh:mm:ss format (time=TIME), and the executable name  
(ucmd=CMD). Output is unsorted by default.  
The use of BSD-style options will add process state (stat=STAT) to the default  
display and show the command args (args=COMMAND) instead of the executable name. You  
can override this with the PS_FORMAT environment variable. The use of BSD-style  
options will also change the process selection to include processes on other  
terminals (TTys) that are owned by you; alternately, this may be described as setting  
the selection to be the set of all processes filtered to exclude processes owned by  
other users or not on a terminal. These effects are not considered when options are  
described as being "identical" below, so -M will be considered identical to Z and so  
on.  
Manual page ps(1) line 1 (press h for help or q to quit)
```

Per quanto riguarda invece **“jobs”**, il comando **“man”** purtroppo non ci restituisce un manuale. Allora dobbiamo prima fare uno switch con la shell **“bash”** e poi utilizzare il comando **“help jobs”** per visualizzare il manuale.

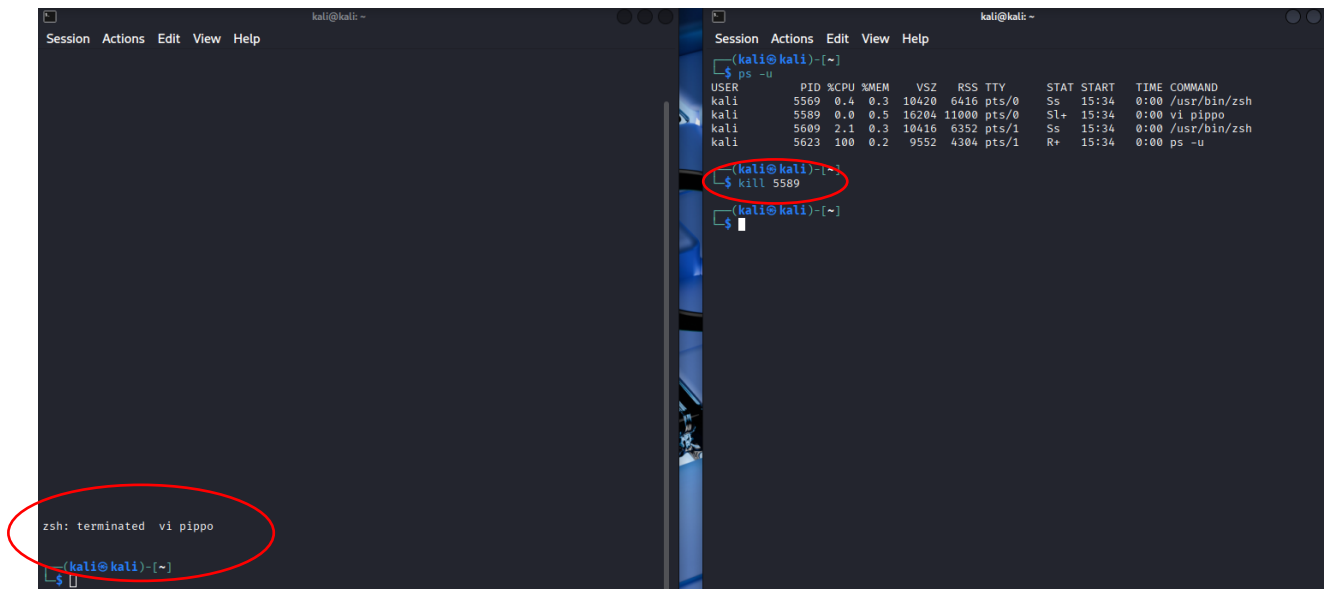
```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ man jobs  
No manual entry for jobs  
(kali@kali)-[~]  
$ bash  
(kali@kali)-[~]  
$ help jobs  
jobs: jobs [-lnprs] [jobspec ...] or jobs -x command [args]  
Display status of jobs.  
  
Lists the active jobs. JOBSPEC restricts output to that job.  
Without options, the status of all active jobs is displayed.  
  
Options:  
-l lists process IDs in addition to the normal information  
-n lists only processes that have changed status since the last  
notification  
-p lists process IDs only  
-r restrict output to running jobs  
-s restrict output to stopped jobs  
  
If -x is supplied, COMMAND is run after all job specifications that  
appear in ARGS have been replaced with the process ID of that job's  
process group leader.  
  
Exit Status:  
Returns success unless an invalid option is given or an error occurs.  
If -x is used, returns the exit status of COMMAND.  
(kali@kali)-[~]  
$
```

Una volta letti i manuali, lanciamo il comando “**vi pippo**” aprendo, così, un nuovo processo, visualizzabile tramite il comando “**ps -u**” su un nuovo terminale con il **PID 5589**.



```
(kali@kali)-[~]
$ ps -u
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
kali      5569   0.4   0.3  10420  6416 pts/0    Ss   15:34   0:00 /usr/bin/zsh
kali      5589   0.0   0.5  16204  11000 pts/0    Sl+  15:34   0:00 vi pippo
kali      5609   2.1   0.3  10416  6352 pts/1    Ss   15:34   0:00 /usr/bin/zsh
kali      5623  100   0.2   9552  4304 pts/1    R+   15:34   0:00 ps -u
```

Ora andiamo a killare (chiudere) il processo “**vi**” in modo da sbloccare il terminale, tramite il comando “**kill**” seguito dal numero **PID 5589**.



```
(kali@kali)-[~]
$ ps -u
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
kali      5569   0.4   0.3  10420  6416 pts/0    Ss   15:34   0:00 /usr/bin/zsh
kali      5589   0.0   0.5  16204  11000 pts/0    Sl+  15:34   0:00 vi pippo
kali      5609   2.1   0.3  10416  6352 pts/1    Ss   15:34   0:00 /usr/bin/zsh
kali      5623  100   0.2   9552  4304 pts/1    R+   15:34   0:00 ps -u

(kali@kali)-[~]
$ kill 5589

(kali@kali)-[~]
$
```

```
zsh: terminated vi pippo
(kali@kali)-[~]
$
```

L'esercizio ci chiede di eseguire Firefox in background utilizzando il comando "**firefox &**" e, successivamente, di portarlo in foreground, utilizzando il comando "**fg**".

Vedremo come Firefox sia attivo (in background) ma all'interno del terminale ci viene permesso di effettuare altri comandi, mentre quando lo andiamo a portare in foreground, il terminale viene occupato da quel processo e non possiamo utilizzare altri comandi prima di terminarlo.

```
kali@kali: ~  
Session Actions Edit View Help  
  
(kali@kali)-[~]  
$ firefox &  
[1] 6628  
  
(kali@kali)-[~]  
$
```

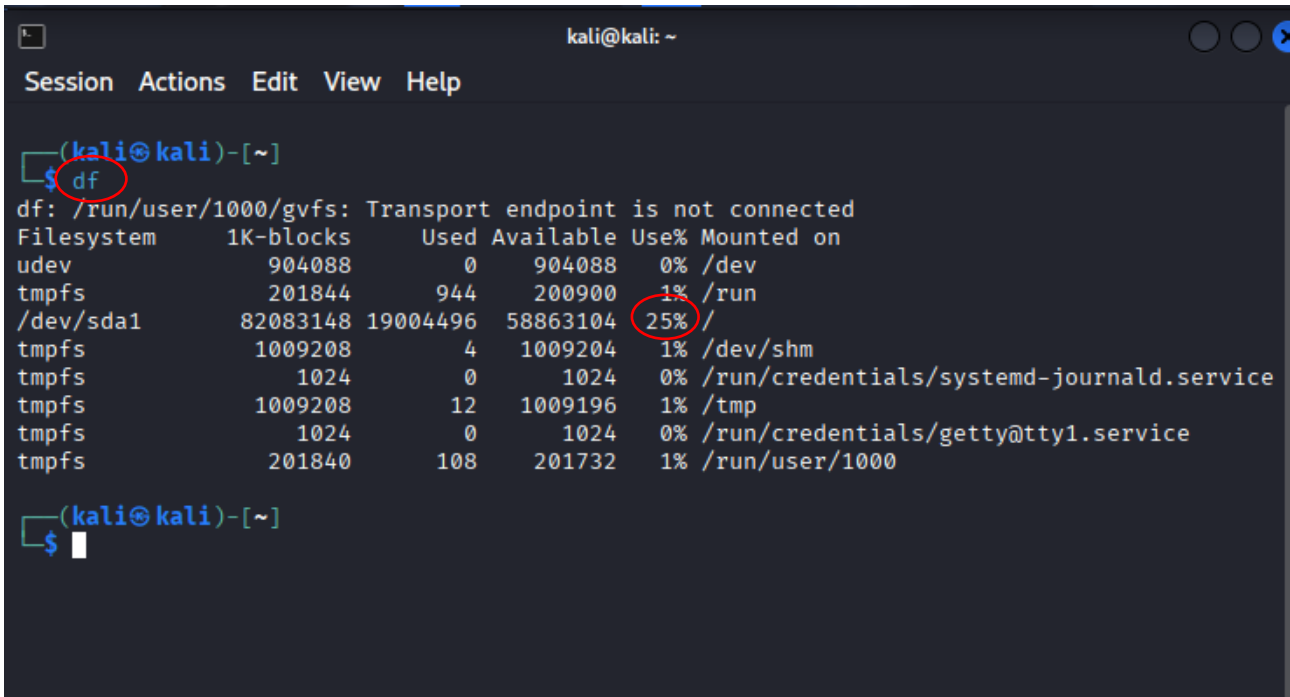
```
kali@kali: ~  
Session Actions Edit View Help  
  
(kali@kali)-[~]  
$ firefox &  
[1] 6628  
  
(kali@kali)-[~]  
$ fg  
[1] + running      firefox  
$
```

Ora, come fatto precedentemente, non ci resta che aprire un altro terminale, visualizzare i processi attivi con "**ps -u**" e terminarlo tramite il comando "**kill**".

```
kali@kali: ~  
Session Actions Edit View Help  
  
(kali@kali)-[~]  
$ firefox &  
[1] 6628  
  
(kali@kali)-[~]  
$ fg  
[1] + running      firefox  
Exiting due to channel error.  
Exiting due to channel error.  
Exiting due to channel error.  
Exiting due to channel error.  
zsh: terminated firefox  
  
(kali@kali)-[~]  
$
```

```
kali@kali: ~  
Session Actions Edit View Help  
  
(kali@kali)-[~]  
$ ps -u  
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND  
kali      5775  0.1  0.3  10428  6572 pts/0    Ss   15:44   0:00 /usr/bin/zsh  
kali      6628  3.9 15.7 2872944 318172 pts/0    SNL+ 15:54   0:00 /usr/bin/firefox-esr/firefox-esr  
kali      6682  0.0  2.1 235432 44116 pts/0    SNL+ 15:54   0:00 /usr/lib/firefox-esr/firefox-esr  
kali      6697  0.4  6.4 2475192 129360 pts/0    SNL+ 15:54   0:00 /usr/lib/firefox-esr/firefox-esr  
kali      6704  0.0  2.5 373344 51580 pts/0    SNL+ 15:54   0:00 /usr/lib/firefox-esr/firefox-esr  
kali      6761  0.3  5.0 2459928 101864 pts/0    SNL+ 15:54   0:00 /usr/lib/firefox-esr/firefox-esr  
kali      6792  0.0  2.0 232820 41624 pts/0    SNL+ 15:54   0:00 /usr/lib/firefox-esr/firefox-esr  
kali      6795  0.2  3.9 2431388 78880 pts/0    SNL+ 15:54   0:00 /usr/lib/firefox-esr/firefox-esr  
kali      6797  0.2  3.8 2431388 78696 pts/0    SNL+ 15:54   0:00 /usr/lib/firefox-esr/firefox-esr  
kali      6843  0.2  3.8 2429312 78624 pts/0    SNL+ 15:54   0:00 /usr/lib/firefox-esr/firefox-esr  
kali      6914  0.2  0.3  10420  6240 pts/1    Ss   15:56   0:00 /usr/bin/zsh  
kali      6931  0.0  0.2  9552  4352 pts/1    R+   15:57   0:00 ps -u  
  
(kali@kali)-[~]  
$ kill 6628  
  
(kali@kali)-[~]  
$
```

Per terminare l'esercizio, siamo andati a verificare quanto spazio si sta occupando, in percentuale, su disco tramite il comando "**df**" (Disk Free)



```
kali@kali: ~  
Session Actions Edit View Help  
  
(kali@kali)-[~]  
$ df  
df: /run/user/1000/gvfs: Transport endpoint is not connected  
Filesystem      1K-blocks      Used Available Use% Mounted on  
udev             904088          0     904088   0% /dev  
tmpfs            201844          944     200900   1% /run  
/dev/sda1        82083148 19004496  58863104  25% /  
tmpfs            1009208           4     1009204   1% /dev/shm  
tmpfs             1024            0         1024   0% /run/credentials/systemd-journald.service  
tmpfs            1009208          12     1009196   1% /tmp  
tmpfs             1024            0         1024   0% /run/credentials/getty@tty1.service  
tmpfs            201840          108     201732   1% /run/user/1000  
  
(kali@kali)-[~]  
$
```

Conclusione:

Questo esercizio mi ha permesso di comprendere come monitorare utenti, processi e risorse di sistema in Linux. Ho imparato a gestire processi in foreground e background, utilizzare ps, fg, bg, kill, e controllare lo spazio su disco con df

In generale, ho acquisito maggiore padronanza del terminale e dei comandi base dell'amministrazione di sistema.

Venezia, 21/10/2025

Fabio Renna